Catedra: Framework empresariales.
Docente: Ricardo Quintanilla.

Ejercicio para trabajar con java servlet y JSTP.

Crear la base de datos semana04, la cual tendrá las tablas clientes, pedidos cada una con los campos que se especifican a continuación.

La tabla de "Clientes" tendrá los siguientes campos:

1. ID: Identificador único de cada cliente (clave primaria)
2. Nombre: Nombre del cliente
3. Dirección: Dirección del cliente
4. Teléfono: Número de teléfono del cliente
5. Email: Dirección de correo electrónico del cliente

La tabla de "Pedidos" tendrá los siguientes campos:

1. ID: Identificador único de cada pedido (clave primaria)
2. ID_Cliente: Identificador del cliente que realizó el pedido (clave foránea)
3. Fecha: Fecha en que se realizó el pedido
4. Total: Total del pedido
5. Estado: Estado actual del pedido

Para crear la aplicación CRUD, utilizaremos Java Servlet y JSTL. Crear las siguientes operaciones CRUD utilizando el modelo MVC:

Acá una les dejo una sugerencia de como crear las estructuras de MVC.

Modelo:

- Crear un nuevo cliente
- Leer los detalles de un cliente existente
- Actualizar los detalles de un cliente existente
- Eliminar un cliente existente
- Crear un nuevo pedido

- Leer los detalles de un pedido existente
- Actualizar los detalles de un pedido existente
- Eliminar un pedido existente

Controlador:

- Procesar la solicitud de crear un nuevo cliente
- Procesar la solicitud de leer los detalles de un cliente existente
- Procesar la solicitud de actualizar los detalles de un cliente existente
- Procesar la solicitud de eliminar un cliente existente
- Procesar la solicitud de crear un nuevo pedido
- Procesar la solicitud de leer los detalles de un pedido existente
- Procesar la solicitud de actualizar los detalles de un pedido existente
- Procesar la solicitud de eliminar un pedido existente

Vista:

- Mostrar el formulario para crear un nuevo cliente
- Mostrar los detalles de un cliente existente
- Mostrar el formulario para actualizar los detalles de un cliente existente
- Confirmar la eliminación de un cliente existente
- Mostrar el formulario para crear un nuevo pedido
- Mostrar los detalles de un pedido existente
- Mostrar el formulario para actualizar los detalles de un pedido existente
- Confirmar la eliminación de un pedido existente

Para implementar esta aplicación, primero debemos diseñar y crear el modelo. A continuación, debemos diseñar y crear el controlador para manejar las solicitudes del usuario y controlar la interacción entre la vista y el modelo. Finalmente, debemos diseñar y crear la vista para presentar la información al usuario y recibir la entrada del usuario.

Resumen de las tareas a realizar para la práctica.

- Crear la base de datos y las tablas utilizando MySQL(MariaDB).
- Crear el modelo para las tablas "Clientes" y "Pedidos".
- Crear el controlador para procesar las solicitudes HTTP y controlar la interacción entre la vista y el modelo.

- Crear la vista para presentar la información al usuario y recibir la entrada del usuario.
- Crear páginas para las vistas, utilizando etiquetas JSTL para interactuar con el controlador.
- Implementar la funcionalidad CRUD en el controlador utilizando el modelo y las páginas JSP.
- Prueba y depuración de la aplicación.

Algunas consideraciones para el desarrollo de la aplicación:

Deben de seguir las mejores prácticas para la seguridad, como validar la entrada del usuario y evitar la inyección de SQL. También es importante diseñar y crear una estructura de proyecto bien organizada para facilitar el mantenimiento y la expansión futura.

Algunas consideraciones a tomar en cuenta para evitar las inyecciones SQL.

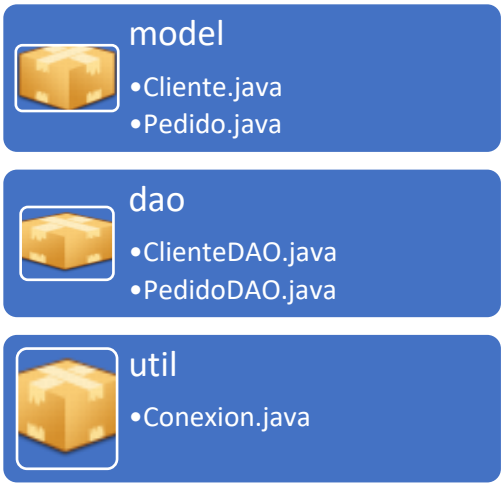
1. Validación de entrada: Validar la entrada del usuario es la forma más importante de prevenir las inyecciones de SQL. La entrada del usuario, como los datos ingresados en un formulario web, debe ser validada en el lado del servidor para asegurarse de que los datos ingresados sean seguros y no contengan código malicioso. La validación de entrada también ayuda a evitar errores y fallas en la aplicación.
2. Utilizar consultas preparadas (prepared statements vistas en el módulo desarrollo de aplicaciones multiplataforma): Las consultas preparadas son una técnica de programación que permite que las consultas SQL se preparen antes de ejecutarlas, lo que ayuda a prevenir las inyecciones de SQL. Las consultas preparadas separan los datos de la consulta SQL, lo que evita que los datos del usuario se interpreten como parte de la consulta SQL.
3. Validación de tipos de datos: La validación de tipos de datos asegura que los datos del usuario sean del tipo de datos esperado y evita que se introduzcan datos maliciosos en el sistema. Por ejemplo, si un campo de número de tarjeta de crédito solo acepta números, cualquier entrada que no sea un número debería ser rechazada.
4. Limitar los permisos de la cuenta de la base de datos: Las cuentas de la base de datos deben tener los permisos mínimos necesarios para realizar las operaciones requeridas por la aplicación. Esto limita el daño que puede causar un atacante si logra obtener acceso a la base de datos.
5. Evitar la concatenación de cadenas SQL: Evita la concatenación de cadenas SQL para construir consultas SQL. En su lugar, utiliza consultas preparadas.
6. Enmascarar los errores de la base de datos: Al mostrar mensajes de error personalizados en lugar de los errores de la base de datos reales, se puede evitar que los atacantes aprendan sobre la estructura y el diseño de la base de datos.

NOTA: Las tareas son sugeridas.

La estructura de los paquetes.

- com.reqp (paquete raíz)
    - model (paquete que contiene las clases que representan los objetos del modelo de datos)
        - Cliente.java
        - Pedido.java
    - controller (paquete que contiene las clases del controlador)
        - ClienteController.java
        - PedidoController.java
    - dao (paquete que contiene las clases de acceso a datos)
        - ClienteDAO.java
        - PedidoDAO.java
    - util (paquete que contiene las clases de utilidad)
        - Conexion.java (clase que gestiona la conexión a la base de datos)
    - view (paquete que contiene las vistas)
        - cliente (paquete que contiene las vistas relacionadas con los clientes)
            - ClienteForm.jsp
            - ClienteList.jsp
            - ClienteEdit.jsp
        - pedido (paquete que contiene las vistas relacionadas con los pedidos)
            - PedidoForm.jsp
            - PedidoList.jsp
            - PedidoEdit.jsp

Código proporcionado.



Códigos de las clases para los modelos:

model/Cliente.java

```
package com. reqp.model;
import java.math.BigDecimal;
import java.util.Date;
public class Cliente {
   private int id;
   private String nombre;
   private String direccion;
   private String telefono;
   private String email;

   public Cliente(int id, String nombre, String direccion, String telefono, String email) {
      this.id = id;
      this.nombre = nombre;
      this.direccion = direccion;
      this.telefono = telefono;
      this.email = email;
   }
   // Métodos getter y setter

   public int getId() {
      return id;
   }

   public void setId(int id) {
      this.id = id;
   }

   public String getNombre() {
      return nombre;
   }
```

```
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getDireccion() {
        return direccion;
    }

    public void setDireccion(String direccion) {
        this.direccion = direccion;
    }

    public String getTelefono() {
        return telefono;
    }
    public void setTelefono(String telefono) {
        this.telefono = telefono;
    }

    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
}
```

model/Pedido.java

```
package com.reqp.model;

import java.math.BigDecimal;
import java.util.Date;

public class Pedido {
    private int id;
    private int idCliente;
    private Date fecha;
    private BigDecimal total;
    private String estado;

    public Pedido(int id, int idCliente, Date fecha, BigDecimal total, String estado) {
        this.id = id;
        this.idCliente = idCliente;
        this.fecha = fecha;
        this.total = total;
        this.estado = estado;
```

```
    }

    // Métodos getter y setter

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public int getIdCliente() {
        return idCliente;
    }

    public void setIdCliente(int idCliente) {
        this.idCliente = idCliente;
    }

    public Date getFecha() {
        return fecha;
    }

    public void setFecha(Date fecha) {
        this.fecha = fecha;
    }

    public BigDecimal getTotal() {
        return total;
    }

    public void setTotal(BigDecimal total) {
        this.total = total;
    }

    public String getEstado() {
        return estado;
    }

    public void setEstado(String estado) {
        this.estado = estado;
    }
}
```

Código de las clases para las clases de DAO

dao/clienteDAO.java

```
package com.reqp.dao;
```

```
import com.reqp.model.Cliente;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class ClienteDAO {
   private Connection con;

   public ClienteDAO(Connection con) {
      this.con = con;
   }

   public List<Cliente> listar() throws SQLException {
      List<Cliente> clientes = new ArrayList<>();
      String sql = "SELECT id, nombre, direccion, telefono, email FROM clientes";
      try (PreparedStatement ps = con.prepareStatement(sql)) {
         try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
               int id = rs.getInt("id");
               String nombre = rs.getString("nombre");
               String direccion = rs.getString("direccion");
               String telefono = rs.getString("telefono");
               String email = rs.getString("email");
               Cliente c = new Cliente(id, nombre, direccion, telefono, email);
               clientes.add(c);
            }
         }
      }
      return clientes;
   }

   public Cliente buscarPorId(int id) throws SQLException {
      Cliente cliente = null;
      String sql = "SELECT id, nombre, direccion, telefono, email FROM clientes WHERE id = ?";
      try (PreparedStatement ps = con.prepareStatement(sql)) {
         ps.setInt(1, id);
         try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
               String nombre = rs.getString("nombre");
               String direccion = rs.getString("direccion");
               String telefono = rs.getString("telefono");
               String email = rs.getString("email");
               cliente = new Cliente(id, nombre, direccion, telefono, email);
            }
         }
      }
      return cliente;
   }

   public void insertar(Cliente cliente) throws SQLException {
      String sql = "INSERT INTO clientes (nombre, direccion, telefono, email) VALUES (?, ?, ?, ?)";
      try (PreparedStatement ps = con.prepareStatement(sql)) {
         ps.setString(1, cliente.getNombre());
```

```
            ps.setString(2, cliente.getDireccion());
            ps.setString(3, cliente.getTelefono());
            ps.setString(4, cliente.getEmail());
            ps.executeUpdate();
        }
    }

    public void actualizar(Cliente cliente) throws SQLException {
        String sql = "UPDATE clientes SET nombre = ?, direccion = ?, telefono = ?, email = ? WHERE id = ?";
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, cliente.getNombre());
            ps.setString(2, cliente.getDireccion());
            ps.setString(3, cliente.getTelefono());
            ps.setString(4, cliente.getEmail());
            ps.setInt(5, cliente.getId());
            ps.executeUpdate();
        }
    }

    public void eliminar(int id) throws SQLException {
        String sql = "DELETE FROM clientes WHERE id = ?";
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);
            ps.executeUpdate();
        }
    }
}
```

dao/PedidoDAO.java

```
pakge com.reqp.dao;


import com.reqp.model.Pedido;
import java.math.BigDecimal;
import java.sql.*;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

public class PedidoDAO {
    private Connection con;
    public PedidoDAO(Connection con) {
        this.con = con;
    }

    public List<Pedido> listar() throws SQLException {
        List<Pedido> pedidos = new ArrayList<>();
        String sql = "SELECT id, id_cliente, fecha, total, estado FROM pedidos";
```

```
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                int id = rs.getInt("id");
                int idCliente = rs.getInt("id_cliente");
                Date fecha = rs.getDate("fecha");
                BigDecimal total = rs.getBigDecimal("total");
                String estado = rs.getString("estado");
                Pedido p = new Pedido(id, idCliente, fecha, total, estado);
                pedidos.add(p);
            }
        }
    }
    return pedidos;
}

public Pedido buscarPorId(int id) throws SQLException {
    Pedido pedido = null;
    String sql = "SELECT id, id_cliente, fecha, total, estado FROM pedidos WHERE id = ?";
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, id);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                int idCliente = rs.getInt("id_cliente");
                Date fecha = rs.getDate("fecha");
                BigDecimal total = rs.getBigDecimal("total");
                String estado = rs.getString("estado");
                pedido = new Pedido(id, idCliente, fecha, total, estado);
            }
        }
    }
    return pedido;
}

public void insertar(Pedido pedido) throws SQLException {
    String sql = "INSERT INTO pedidos (id_cliente, fecha, total, estado) VALUES (?, ?, ?, ?)";
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, pedido.getIdCliente());
        ps.setDate(2, new java.sql.Date(pedido.getFecha().getTime()));
        ps.setBigDecimal(3, pedido.getTotal());
        ps.setString(4, pedido.getEstado());
        ps.executeUpdate();
    }
}

public void actualizar(Pedido pedido) throws SQLException {
    String sql = "UPDATE pedidos SET id_cliente = ?, fecha = ?, total = ?, estado = ? WHERE id = ?";
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, pedido.getIdCliente());
        ps.setDate(2, new java.sql.Date(pedido.getFecha().getTime()));
        ps.setBigDecimal(3, pedido.getTotal());
        ps.setString(4, pedido.getEstado());
        ps.setInt(5, pedido.getId());
```

```
            ps.executeUpdate();
        }
    }

    public void eliminar(int id) throws SQLException {
        String sql = "DELETE FROM pedidos WHERE id = ?";
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);
            ps.executeUpdate();
        }
    }
}
```

Código de las clases para las clases de util

util/Conexion.java

```
package util;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class Conexion {
    private static final String URL = "jdbc:mysql://localhost:3306/semana04";
    private static final String USUARIO = "root";
    private static final String CLAVE = "";

    public static Connection getConexion() throws SQLException {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            return DriverManager.getConnection(URL, USUARIO, CLAVE);
        } catch (ClassNotFoundException ex) {
            throw new SQLException("No se pudo cargar el driver de la base de datos");
        }
    }
}
```

Código para crear la base de datos.

```
Autocomplete: [Tab]->Next Tag. [Ctrl+Space]->List All Tags. [Ctrl+Enter]->List Matching Tags. [C
 1    CREATE DATABASE semana04;
 2
 3    USE semana04;
 4
 5   CREATE TABLE clientes (
 6      id INT NOT NULL AUTO_INCREMENT,
 7      nombre VARCHAR(50) NOT NULL,
 8      direccion VARCHAR(100) NOT NULL,
 9      telefono VARCHAR(15) NOT NULL,
10      email VARCHAR(100) NOT NULL,
11      PRIMARY KEY (id)
12    );
13
14   CREATE TABLE pedidos (
15      id INT NOT NULL AUTO_INCREMENT,
16      id_cliente INT NOT NULL,
17      fecha DATE NOT NULL,
18      total DECIMAL(10, 2) NOT NULL,
19      estado VARCHAR(20) NOT NULL,
20      PRIMARY KEY (id),
21      FOREIGN KEY (id_cliente) REFERENCES clientes(id)
22    );
```

Crear usuarios en mariaDB

Crear usuario caleb con clave dominic con todos los privilegios sobre la base de datos de semana04.

Crear el usuario dominic con clave 123456 con los permisos de solo lectura sobre la base de datos semana04.

```
CREATE USER 'caleb'@'%' IDENTIFIED BY 'tu_clave';
GRANT ALL PRIVILEGES ON *.* TO 'caleb'@'%';


CREATE USER 'dominic'@'%' IDENTIFIED BY 'tu_clave';
GRANT SELECT ON *.* TO 'dominic'@'%';
```